



Departamento de Programación
Facultad de Informática
Universidad Nacional del Comahue



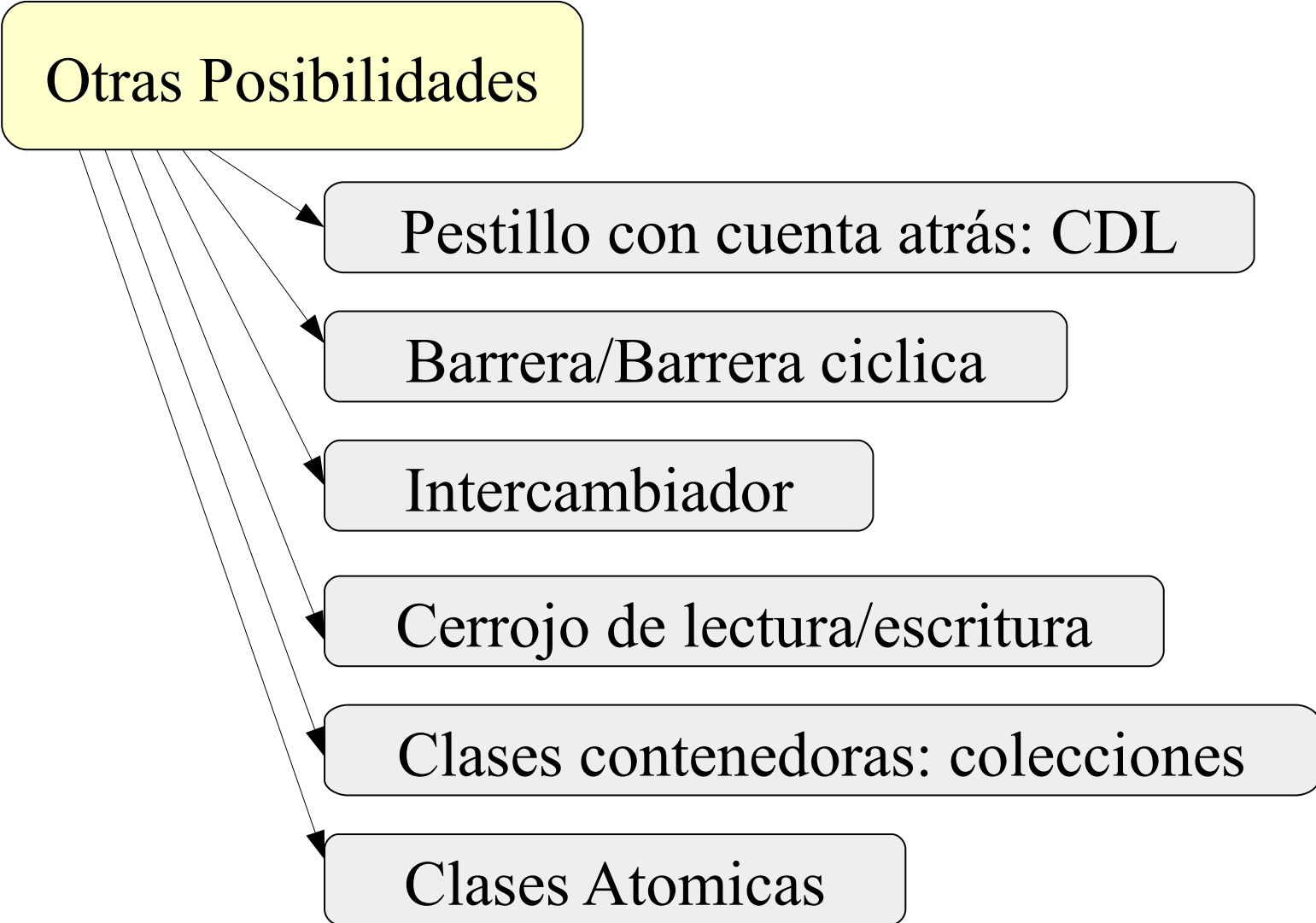
Programación Concurrente



Otras facilidades para concurrencia

Sincronización

Otras Posibilidades



```
graph TD; A[Otras Posibilidades] --> B[Pestillo con cuenta atrás: CDL]; A --> C[Barrera/Barrera ciclica]; A --> D[Intercambiador]; A --> E[Cerrojo de lectura/escritura]; A --> F[Clases contenedoras: colecciones]; A --> G[Clases Atomicas];
```

The diagram illustrates various synchronization techniques. A central yellow box labeled 'Otras Posibilidades' has six arrows pointing to a vertical list of gray boxes: 'Pestillo con cuenta atrás: CDL', 'Barrera/Barrera ciclica', 'Intercambiador', 'Cerrojo de lectura/escritura', 'Clases contenedoras: colecciones', and 'Clases Atomicas'. On the far left, there is a vertical stack of five teal diamond shapes. On the top right, there is a teal L-shaped decorative element.

Pestillo con cuenta atrás: CDL

Barrera/Barrera ciclica

Intercambiador

Cerrojo de lectura/escritura

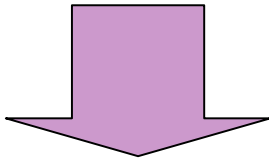
Clases contenedoras: colecciones

Clases Atomicas

Sincronización de varios hilos

Esperar por un hilo

- Se puede utilizar el método *join()*



- Esperar por muchos hilos nos implicaría hacer muchas llamadas *join()*, una por cada hilo activo.
- Para que comiencen varios hilos a la vez o para esperar que todos ellos terminen ..

- // Arranque de hilo
- **hilo.start();**
- ...
- // Espera a que termine
- **hilo.join();**





Pestillo con cuenta atrás

- Mecanismo de sincronización que permite que un conjunto de hilos esperen a un contador que debe llegar a 0
- Es un objeto que se inicializa con un contador N que podemos ir decrementando uno a uno.
- El N indica la cantidad de tareas que pondremos a esperar.
- Cuando el contador llega a cero, se liberan todas las tareas que esperan.
- Son útiles cuando un conjunto fijo de hilos deben esperarse para una tarea en común.
- Operaciones:
 - De espera
 - De decremento

Pestillo con cuenta atrás

- En Java se llaman **CountDownLatch**
- El objeto CountDownLatch libera los hilos al llegar el contador a 0.
- Operaciones:
 - await()
 - await(timeout, UNIT)
 - countDown()

```
import java.util.concurrent.CountDownLatch;
```

Pestillo con cuenta atrás

- Constructor
`CountDownLatch (n)`
- `n` indica la cantidad de veces que debe decrementarse el contador (ejecutarse la operación “countDown”) para que los hilos es espera (por un “await”) puedan continuar
- Ejemplo:
empezar: `countDownLatch(1)`
terminar: `countDownLatch(n)`

Pestillo con cuenta atrás

- Ejemplo:

empezar: CountdownLatch(1)

hilo1 -----> empezar.await();

...

hilon -----> empezar.await();

main -----> empezar.countDown();

terminar: countdownLatch(n)

hilo1 -----> terminar.countDown();

...

hilon -----> terminar.countDown();

main -----> terminar.await();



Barrera

- Mecanismo de sincronización que permite que un conjunto de hilos esperen a llegar a un punto de barrera
- Una barrera de N posiciones retiene los primeros $N-1$ hilos que llegan. Cuando llega el enésimo, permite que salgan todos. La barrera se rompe y nuevos hilos pasan sin esperar.
- También se puede trabajar con Barrera Ciclica

Barrera cíclica

Variante de Barrera

- En Java se llaman **CyclicBarrier**
- Soporta un comando `run()` por punto de corrida
- El objeto barrera libera los hilos al llegar a la cantidad indicada en el constructor.
- Son útiles cuando un conjunto fijo de hilos deben esperarse para una tarea en común y deben sincronizarse repetidamente.
- La barrera es cíclica porque puede ser reutilizada después que los hilos en espera se liberan.
- Se puede considerar que un `CountDownLatch` es una barrera NO cíclica



CyclicBarrier

- Esta clase se instancia pasándole en el constructor cuántos hilos debe sincronizar.

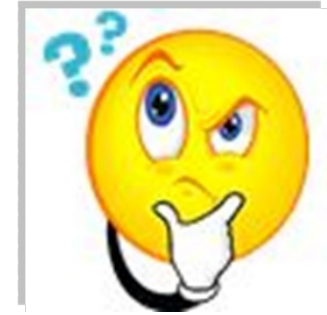
```
//sincroniza 3 hilos  
CyclicBarrier barrera = new CyclicBarrier(3);
```

- Los hilos deben llamar al método `await()` para esperar por los demás hilos que deben llegar a la barrera

```
public void run () {  
    try {  
        ...  
        //Se queda bloqueado hasta que los 3 hilos hagan esta llamada  
        barrera.await();  
        ...  
    } catch (Exception e) { ... }  
    //código del hilo  
}
```

```
import java.util.concurrent.CyclicBarrier;
```

Utilización de barreras



```
Import  
java.util.concurrent.CyclicBarrier;
```

```
CyclicBarrier cb = new CyclicBarrier(n)
```

- **await()** : Espera a que todos los hilos definidos hayan entrado a la barrera
- **int await(long timeout, TimeUnit unit)** : Espera a que todos los hilos definidos hayan entrado a la barrera o que pase el tiempo estipulado
- **int getNumberWaiting()** : Retorna el número de hilos que están esperando en la barrera
- **int getParties()** : Retorna el número de hilos que requiere esta barrera .
- **boolean isBroken()** : Pregunta si la barrera está quebrada
- es verdadero cuando uno o mas hilos rompen la barrera por interrupción o timeout, o por un reset, o la acción de la barrera falla debido a una excepción.
- **void reset()** : Restablece la barrera al estado inicial



Constructores

Barrera Ciclica

```
CyclicBarrier(int cantHilosSincronizados)
```

```
CyclicBarrier  
(int cantHilosSinc, Runnable accionesBarrera)
```

- Una barrera es un punto de espera a partir del cuál todos los hilos se sincronizan
 - Ningún hilo pasa la barrera hasta que todos los hilos esperados llegan a ella
 - **Utilidad: sirve para**
 - Unificar resultados parciales
 - Como inicio a la siguiente fase de ejecución simultánea



Intercambiador (*exchanger*)

- Actúa como un canal síncrono (buffer sin espacio), pero solo soporta un método, tipo *rendezvous*, que combina los efectos de *agregar* y *sacar* del buffer
- La operación, llamada “*exchange*” toma un argumento que representa el objeto ofrecido por un hilo a otro, y retorna el objeto ofrecido por el otro hilo

Dos agentes A y B se sincronizan en 1 punto. A le pasa un dato a B; B le pasa un dato a A.



Intercambiador

- Clase *Exchanger* en Java

```
import java.util.concurrent.Exchanger;
```



Clases *Contenedoras*

Versiones sincronizadas de las principales clases contenedoras
Disponibles en `java.util.concurrent`

Clases:

`ConcurrentHashMap`, `CopyOnWriteArrayList`
`ReadWriteLock`

Colas Sincronizadas – Interfaz `BlockingQueue`

Clases:

LinkedBlockingQueue, *ArrayBlockingQueue*,
SynchronousQueue, *PriorityBlockingQueue*, y
DelayQueue

`import java.util.concurrent. ...`(la clase que corresponda)